

Introduction au Deep LEarning

Architectures de réseaux de neurones

Olivier Schwander

olivier.schwander@sorbonne-universite.fr

Master MIND - Sorbonne Université

2025-2026

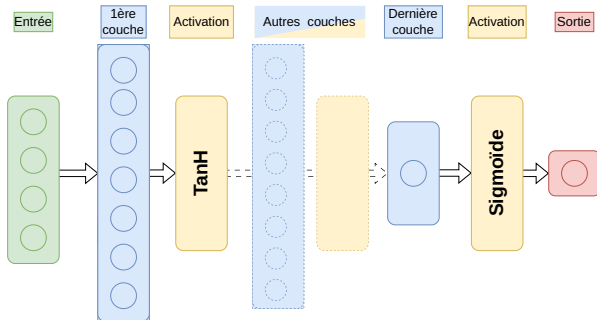
Slides de N. Baskiotis et V. Guigue

Plan

- 1 Tâches et fonctions de coût
- 2 Autres paradigmes
- 3 La(es) révolution(s) Deep Learning

Réseau MLP : Une architecture passe-partout

Un réseau MLP est une succession de couches linéaires et de fonctions d'activation

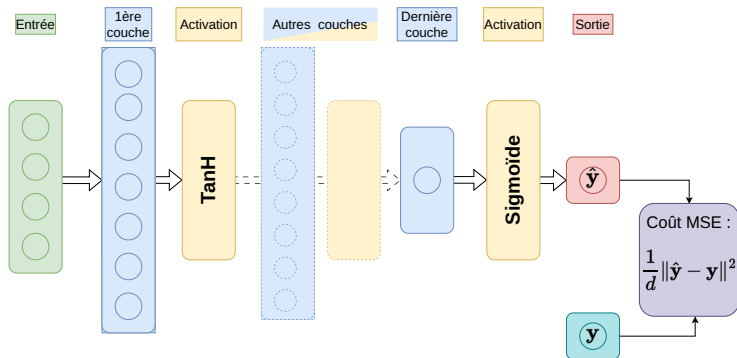


Propriétés du Multi-Layer Perceptron (MLP)

- Idéal pour les tâches simples (régression, classification, multi-classes ...)
- Architecture que l'on retrouve quasiment dans toutes les autres architectures
- Mais non adaptée sur des entrées complexes (texte, image)
- Très sujet au sur-apprentissage avec l'augmentation du nombre de couches

⇒ **La généricité de l'architecture : une histoire de coût !**

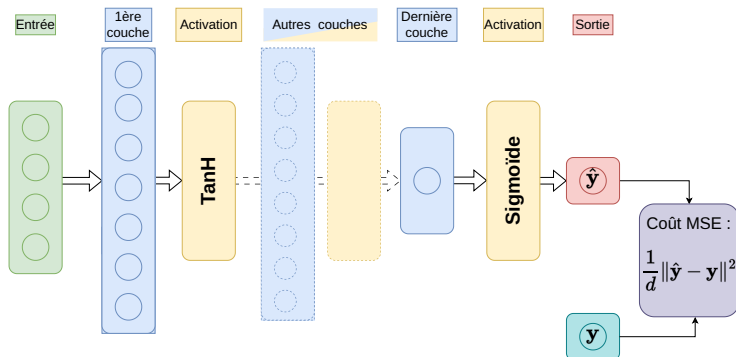
Régression : coût MSE



Coût Moindres Carrés

- Usuel dans le contexte de la régression
- Adapté également aux sorties multi-variées
- **Attention** : choisir une activation de sortie adaptée !
 - ▶ si normalisation entre 0 et 1 → sigmoïde
 - ▶ si normalisation entre -1 et 1 → tangente hyperbolique

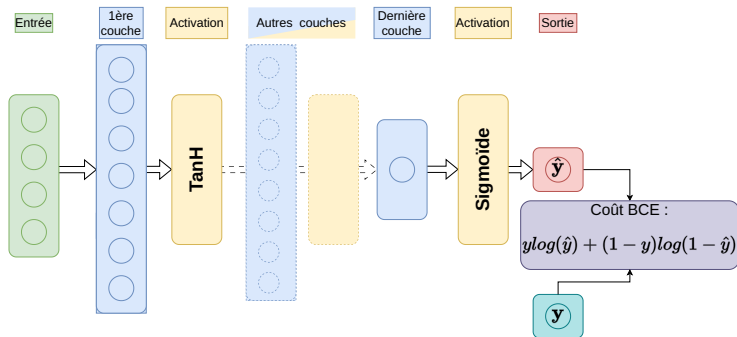
Classification binaire (1) : coût MSE



La MSE fonctionne également !

- Codage soit $\{0, 1\}$ des classes, soit $\{-1, 1\}$
- Seuil à 0.5 pour départager entre les deux classes (ou autre pour régler le compromis rappel/précision)
- Mais pas idéal, le coût n'est pas assez "piqué" vers les extrêmes

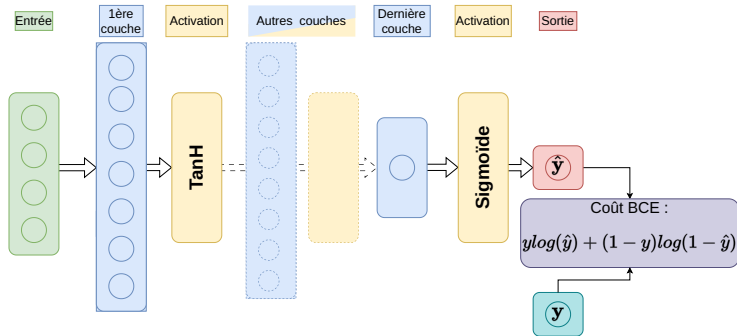
Classification binaire (2) : coût BCE



Binary Cross Entropy

- Codage soit $\{0, 1\}$ des classes
- Sigmoïde comme dernière couche d'activation (la sortie représente la probabilité de la classe 1)
- Seuil à 0.5 pour départager entre les deux classes (ou autre pour régler le compromis rappel/précision)
- Encourage les valeurs extrêmes 0 et 1.

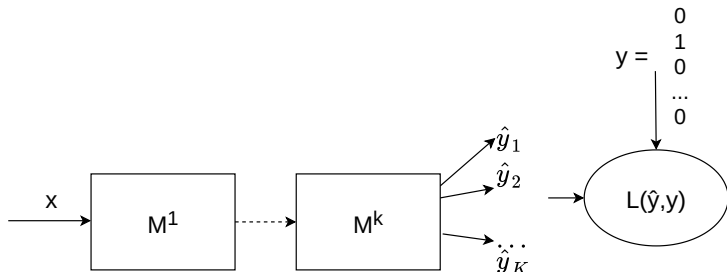
Classification binaire (2) : coût BCE



Binary Cross Quoi ?

- Hypothèse : distribution binomiale sur la sortie :
 - Soit $\hat{y} = f_{\theta}(x)$ la sortie du réseau, qui modélise $p(y = 1|x) = f^*(x)$
- $\Rightarrow P(y|x) = f^*(x)^y (1 - f^*(x))^{1-y}$
- Neg-Log-vraisemblance :
 $NLL((x, y); \theta) = -y \log(f_{\theta}(x)) - (1 - y) \log(1 - f_{\theta}(x)) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$
 - Equivalent Cross-Entropie : $H(p, q) = -\sum_{y \in \mathcal{Y}} p(y) \log(q(y))$, avec p la distribution des "vrais" labels et q la distribution des labels prédits

Multi-classe



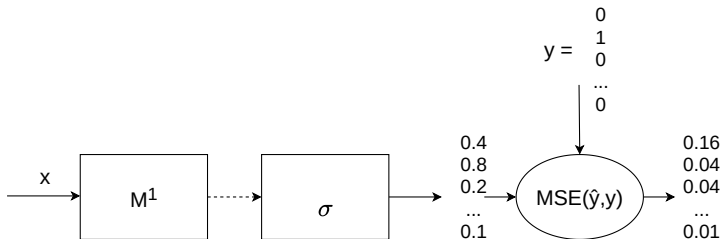
Quand il faut prédire K classes

- K sorties
- Utilisation de vecteurs 1-hot pour la supervision:

$$y = (0, 0, \dots, 1, \dots, 0)$$

avec $y_i = 0$ pour i différent de la bonne classe,
 $y_k = 1$ pour k l'indice de la bonne classe.

Utilisation de la MSE



Fonction de coût problématique

- Sortie du réseau entre 0 et 1 $\Rightarrow$ utilisation d'une sigmoïde
- Mais :
 - ▶ La similarité au vecteur de sortie n'est pas le plus important, c'est l'argmax qui nous intéresse le plus
 - ▶ Pas plus d'efforts mis sur la maximisation de la sortie de la bonne classe que la minimisation des autres sorties

Coût Cross-entropique

SoftMax : Transformer les sorties en distribution

$$\text{SoftMax}(z)_i = e^{z_i} / \left(\sum_{j=1}^K e^{z_j} \right), \quad \sum_{i=1}^K \text{SoftMax}(z)_i = 1$$

Coût Cross-entropique Multi-Classe

- $\text{CE}(y, \hat{y}) = - \sum_{i=1}^K y_i \log(\hat{y}_i)$
- Dans le cas où y est un vecteur one-hot de la classe k : $\text{CE}(y, \hat{y}) = -\log(\hat{y}_k)$
- Identique à la log-vraisemblance et une hypothèse multinomiale sur les sorties
- Combinaison SoftMax et Cross-entropie :

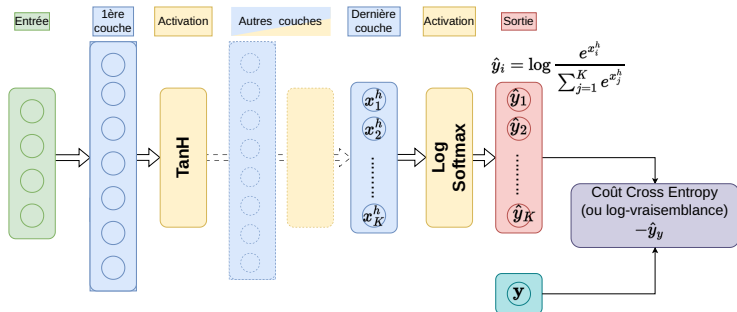
$$\text{CE}(y, \text{SoftMax}(z)) = -z_k + \log \left(\sum_{j=1}^K e^{z_j} \right), \quad \frac{\partial \text{CE}(y, \text{SoftMax}(z))}{\partial z_i} = \text{Softmax}(z)_i - 1_{i=k}$$

Cross-entropie binaire

Pour le multi-class multi-label en particulier, cross-entropie sur chaque sortie de manière

indépendante: $\text{BCE}(y, \hat{y}) = - \sum_{i=1}^K y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)$

Classification multi-classes : coût Cross Entropy



Problème à K classes

- K sorties, chacune représente la probabilité de la classe correspondante
- Softmax sur les sorties pour obtenir une distribution de probabilité
- La classe prédite correspond à l'index de la sortie la plus grande
- Coût Cross Entropy (équivalent log-vraisemblance) pour l'apprentissage
- Si multi-labels, alors utilisation de la BCE sur chaque sortie (hypothèse de sorties indépendantes les unes des autres)

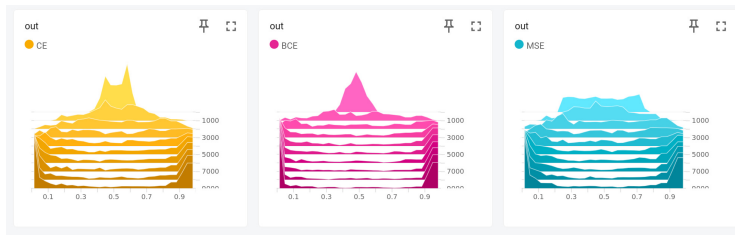
En résumé

La fonction de coût

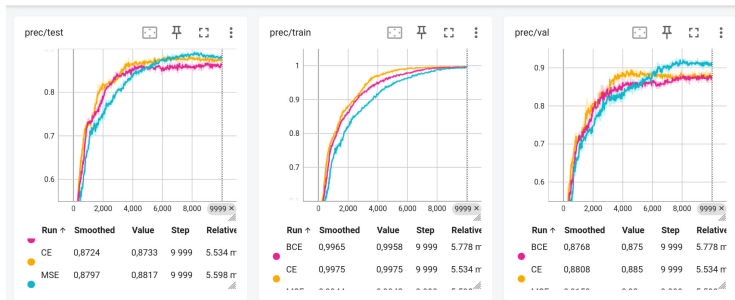
- Essentielle au bon apprentissage, énormément de variantes
- Tâche-dépendante, données-dépendante, très flexible (pondération des classes par exemple pour les données déséquilibrées)

<code>nn.L1Loss</code>	error (MAE) between each element in the input x and target y .	<code>nn.GaussianNLLLoss</code>	Gaussian negative log likelihood loss.	<code>nn.MultiLabelMarginLoss</code>	classification input x (a 2D Tensor) and target y (a 2D Tensor) is a 2D Tensor
<code>nn.MSELoss</code>	Creates a criterion that measures the mean squared error (squared L2 norm) between each element in the input x and target y .	<code>nn.KLDivLoss</code>	The Kullback-Leibler divergence loss.	<code>nn.HuberLoss</code>	Creates a criterion that measures the absolute error between each element in the input x and target y . delta-scaled
<code>nn.CrossEntropyLoss</code>	This criterion computes the cross entropy loss between input logits and target.	<code>nn.BCELoss</code>	Creates a criterion that measures the Binary Cross Entropy between the target and the input probabilities	<code>nn.SmoothL1Loss</code>	Creates a criterion that measures the absolute error between each element in the input x and target y . term other
<code>nn.CTCLoss</code>	The Connectionist Temporal Classification loss.	<code>nn.BCEWithLogitsLoss</code>	This loss combines a <i>Sigmoid</i> layer and the <i>BCELoss</i> in one single class.	<code>nn.SoftMarginLoss</code>	Creates a criterion that measures the loss given inputs x_1 , x_2 , two 1D mini-batch or 0D Tensors, and a label 1D mini-batch or 0D Tensor y (containing 1 or -1).
<code>nn.NLLLoss</code>	The negative log likelihood loss.	<code>nn.MarginRankingLoss</code>	Creates a criterion that measures the loss given inputs x_1 , x_2 , two 1D mini-batch or 0D Tensors, and a label 1D mini-batch or 0D Tensor y (containing 1 or -1).	<code>nn.MultiLabelSoftMarginLoss</code>	Creates a criterion that measures the loss given an input tensor x and a labels tensor y (containing 1 or -1).
<code>nn.PoissonNLLLoss</code>	Negative log likelihood loss with Poisson distribution of target.	<code>nn.HingeEmbeddingLoss</code>	Measures the loss given an input tensor x and a labels tensor y (containing 1 or -1).		

Exemple : Echequier



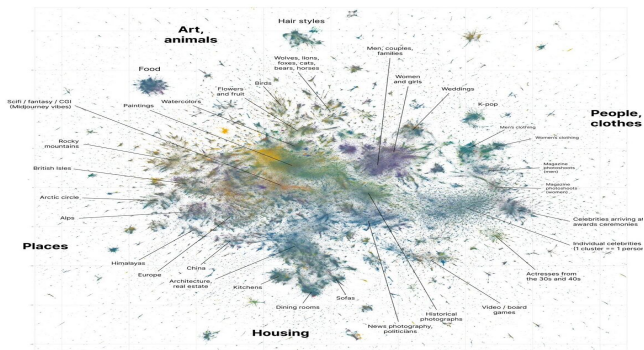
prec 3 cards



Plan

- 1 Tâches et fonctions de coût
- 2 Autres paradigmes
- 3 La(es) révolution(s) Deep Learning

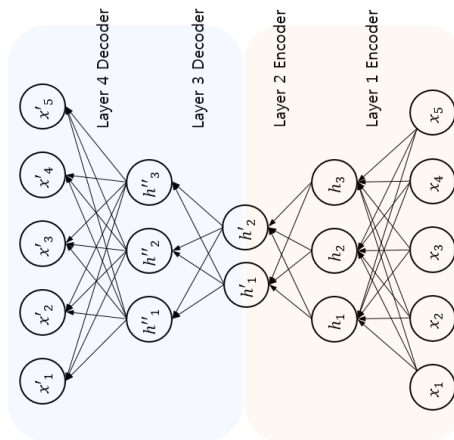
Quand on n'a pas de features ...



Il faut pouvoir projeter des objets discrets dans un espace continu

- Projection aléatoire : a existé (feature hashing), peu efficace
- Pas de description des objets, mais il existe des relations entre eux !
- Un "bon" espace de représentation doit pouvoir exprimer la sémantique latente de ces relations
- Peut être guidé par l'expert (avant le Deep), la tâche (apprentissage end-to-end) ou par les données sans supervision (self-supervised par exemple)

Exemple (1) : Auto-encoders



Apprentissage non-supervisé

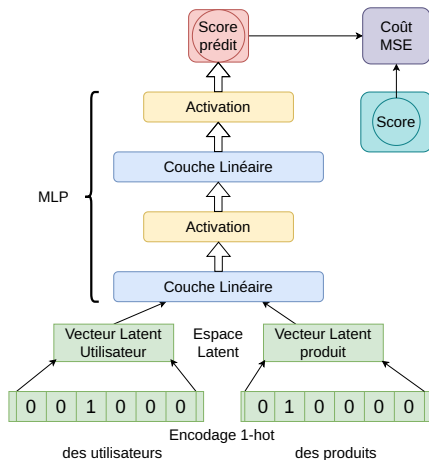
Objectif :

- apprendre une “compression” des données utile pour l'apprentissage
- $f^{-1}(f(x)) \approx x$: la sortie doit être proche de l'entrée

Intérêts :

- Représentation plus adéquate
- clustering des données
- lissage/débruitage
- visualisation
- ..

Exemple (2) : Représentation et recommandation



Données

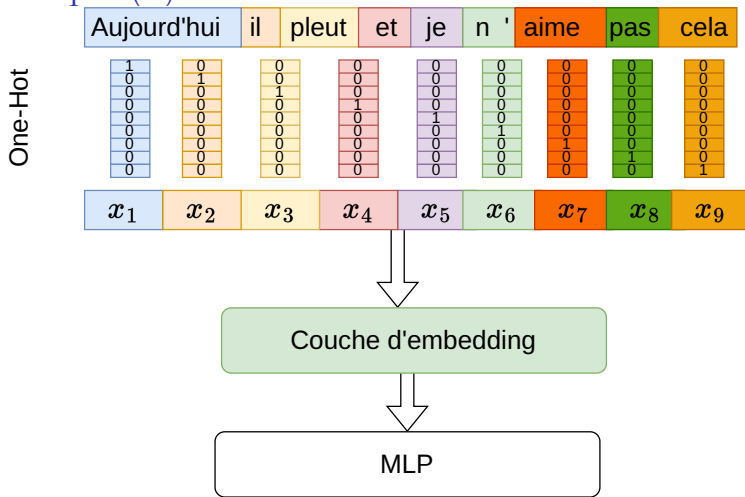
- Des produits (sans description)
- Des utilisateurs (sans description)
- Des notes des utilisateurs sur les produits

Objectif : prédire les avis des utilisateurs

Procédure :

- Encodage one-hot des produits et des utilisateurs (un produit et un utilisateur par dimension)
- Représentation aléatoire initialement
- La back-propagation corrige le MLP et les représentations utilisateurs/produits

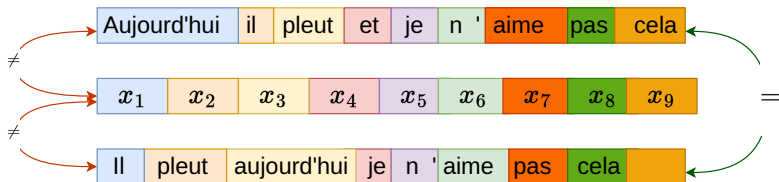
Exemple (3) : le texte



Même objectif :

- Trouver une représentation "utile" pour les éléments atomiques (token)
- Problème de la tokenisation

Exemple (3) : le texte

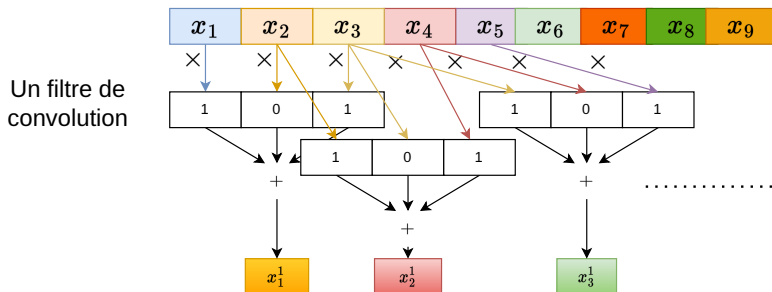


Pendant le MLP n'est plus du tout adapté

- Pour un MLP, chaque dimension a son propre sens
 - Pas de possibilités de permuter les dimensions, de définir des permutations
- ⇒ Impossible de traiter les invariances
- ⇒ Impossible de traiter des longueurs variables

Architecture convolutionnelle 1D

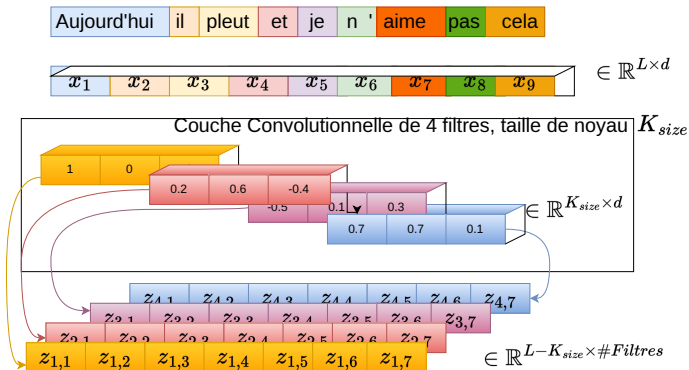
Aujourd'hui il pleut et je n' aime pas cela



Principe d'une convolution

- Filtre qui effectue une somme pondérée sur une petite fenêtre des entrées
- Les poids du filtre détermine la fonction du filtre
- Le filtre est passé itérativement sur toute la séquence et produit une sortie scalaire à chaque position
- Il permet de "détecter" la présence d'une caractéristique à une position

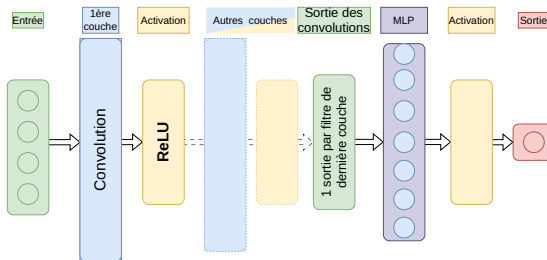
Architecture convolutionnelle 1D



Caractéristiques d'une convolution

- Taille du noyau : taille de la fenêtre glissante
- Stride : déplacement du noyau à chaque itération
- La convolution peut être définie en 1 dimension, en 2, ... tout dépend du type de signal en entrée (texte, image, vidéo, séries temporelles)

Architecture convolutionnelle 1D



Utilisation pour de la classification

- Série de convolutions qui permettent de repérer la présence ou l'absence de certaines caractéristiques linguistiques
- Pour obtenir un vecteur de dimension fixe, on peut prendre le maximum de chaque filtre de la dernière couche (max-pooling global).
- La sortie des couches convolutionnelles est donc un vecteur de taille le nombre de filtres de la dernière couche, chaque filtre spécialisé pour une caractéristique particulière
- Utilisation d'un MLP pour la tâche subséquente.

Convolution 2D

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0



1	0	1
0	1	0
1	0	1

5 x 5 – Image Matrix

3 x 3 – Filter Matrix

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	4

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	4
2		

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	4
2	4	

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	4
2	4	3

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	4
2	4	3
2		

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	4
2	4	3
2	3	

Convolved Feature

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4	3	4
2	4	3
2	3	4

Convolved Feature

Application d'une transformation linéaire sur toutes les régions de l'image

Couche de Pooling

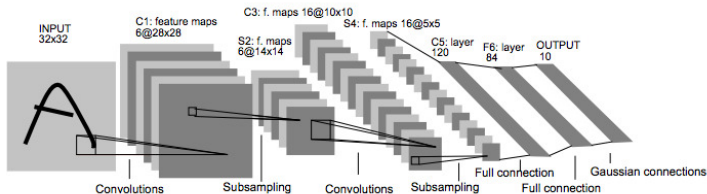


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Pooling (ou subsampling) :
Réduire la dimensionalité de
sortie

- Max Pooling : on prend le max sur une fenêtre
- Average Pooling : on fait la moyenne
- Sum Pooling : la somme ...

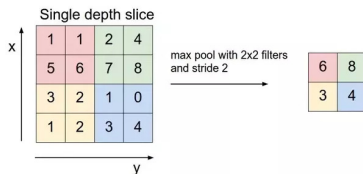
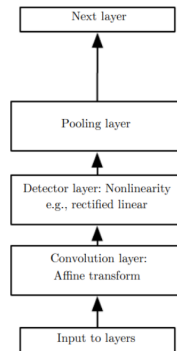
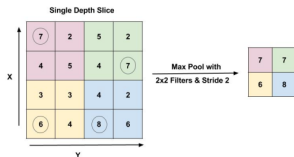
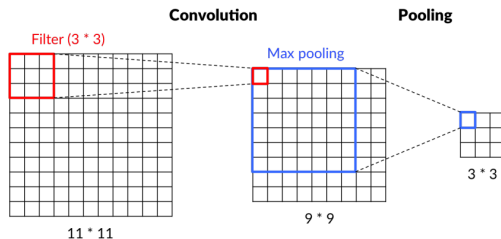
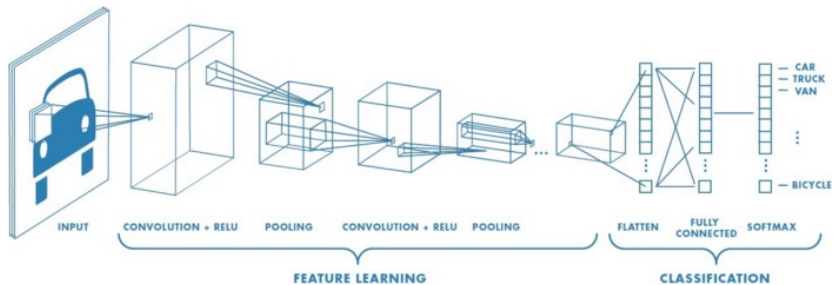


illustration :
<https://medium.com/@RaghavPrabhu/understanding-of-convolutional-neural-network-cnn-deep-learning-99760835f148>

Couche convolutionnelle usuelle



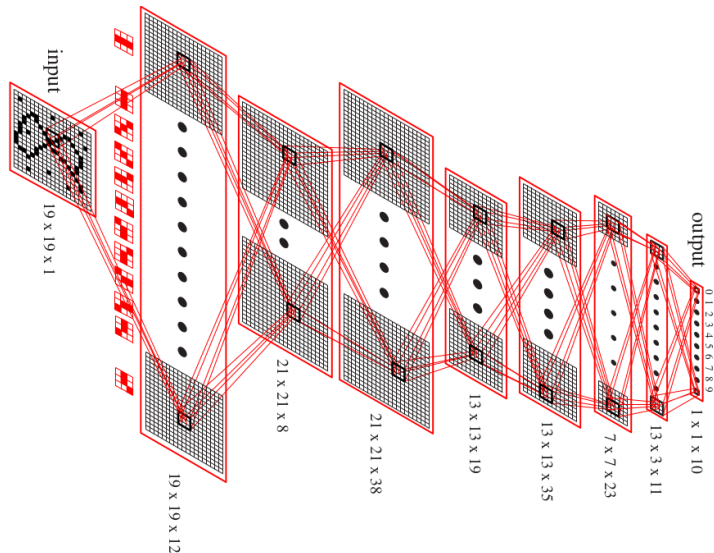
Réseaux convolutifs



Exemple

Reconnaissance de caractères

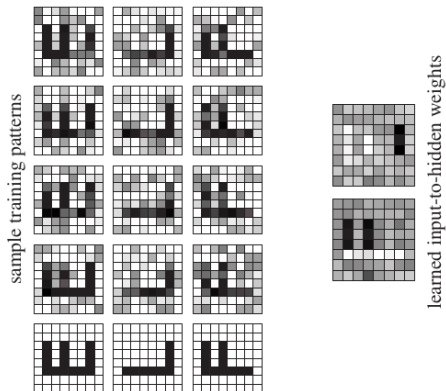
[Duda et al 00]



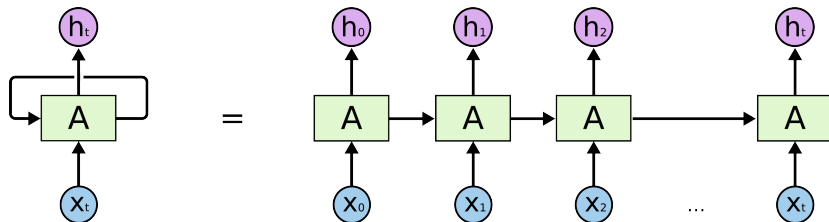
Exemple

Reconnaissance de caractères (couches internes)

[Duda et al 00]



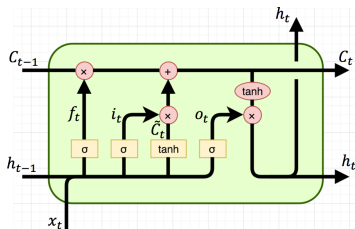
Réseaux récurrents



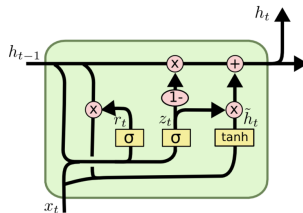
Pour les séquences

- L'objectif est de prédire l'élément suivant d'une séquence : $(x^0, x^1, \dots, x^t)$
- Hypothèse : $x^{t+1} = f(x^t, h^t)$ l'élément suivant d'une séquence dépend des éléments précédents et d'un état mémoire latent.
- Le réseau prédit pour chaque passage l'état h^t latent.
- Un autre réseau est utilisé pour "décoder" l'état mémoire en la valeur x^t associée.

Long Short Term Memory (LSTM) et Gated Recurrent Unit (GRU)



(a) Long Short-Term Memory



(b) Gated Recurrent Unit

Deux variantes des RNNs

- Un processus de mémoire mis-à-jour automatiquement
- Permet de se "souvenir" (d'avoir une information persistente d'état en état)

Et beaucoup d'autres

- RBMs (Restricted Boltzmann Machine)
- V.A.E. (Variational Auto-encoder)
- G.A.N. (Generative Adversial Network)
- Mécanisme d'attention (globale et self-attention)
- Stochastic Unit (Reinforce, Gumbel-Softmax, Straight-Through estimator)

Plan

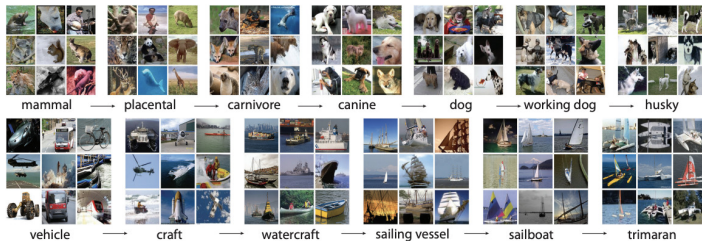
- 1 Tâches et fonctions de coût
- 2 Autres paradigmes
- 3 La(es) révolution(s) Deep Learning

Quelques révolutions du Deep Learning - 1

ImageNet

- Challenge de classification d'images 2009
- Arrivée du Deep en 2012 - écrase la concurrence
- Emergences des modèles les plus connus : AlexNet, Inception, ResNet, ViT...

Résultats depuis 2010

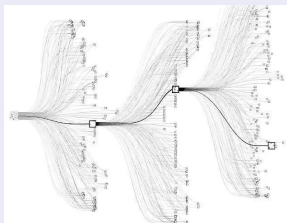


Quelques révolutions du Deep Learning - 2

Deep Q-Learning/Alpha[Go/Zero]

- Capable de jouer à des jeux génériques sans connaître les règles
- 2015 : bat pour la première fois un joueur professionnel de Go
- 2017 : bat le champion du monde de Go

Arbre d'exploration

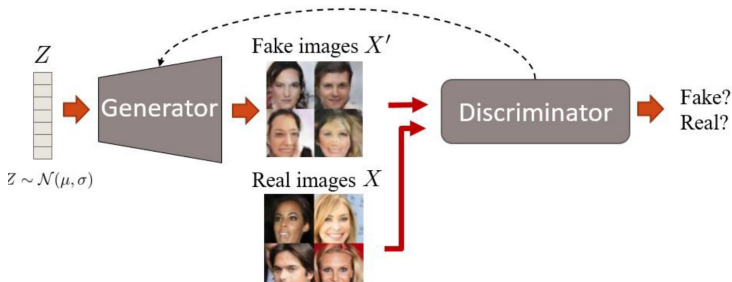
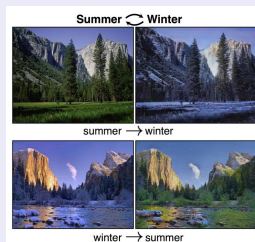


Quelques révolutions du Deep Learning - 3

Generative Adversarial Network (2014)

- Nouveau paradigme d'apprentissage : faussaire contre policier
- Enormément de domaines d'application, en particulier la génération d'image
- Capable d'apprendre des caractéristiques latentes

Transfert de style



Quelques révolutions du Deep Learning - 4

Modèles de Langues

- Origine : word2vec (2013) (apprentissage de représentation), explosion des performances avec le concept d'attention (Transformer, 2017)
- Ont révolutionné tout en TAL : traduction, ChatBot, Recherche d'Information, ...
- Multiples modèles : GPT, BERT, ELMO, Galactica

ChatGPT chatbot



What is brandolini's law?



Brandolini's law, also known as the "Bullshit Asymmetry Principle," is a humorous aphorism that states: "The amount of energy needed to refute bullshit is an order of magnitude bigger than to produce it." In other words, it takes more effort and time to debunk a false or misleading statement than it does to make the statement in the first place.

The law is named after Alberto Brandolini, an Italian software developer who coined the term in a tweet in 2013. Brandolini's law is often used to describe the challenges of debunking misinformation or false claims, particularly in the age of social media, where false and misleading information can spread quickly and easily.

While Brandolini's law is not a formal scientific principle, it is a useful way of illustrating the challenges of combating misinformation and the importance of carefully evaluating information before accepting it as true.

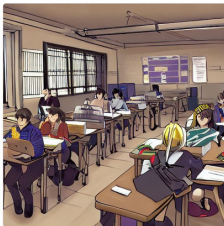
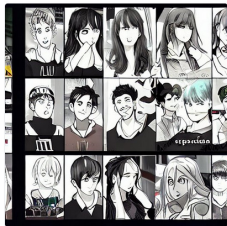


Et quand on mélange toutes ces révolutions ...

On obtient DALL-E, Stable Diffusion et autres modèles du genre

classroom of college students, sorbonne university,| trending in artst

Generate image



a friendly robot with kids, trending in artstation, picasso style

Generate image

